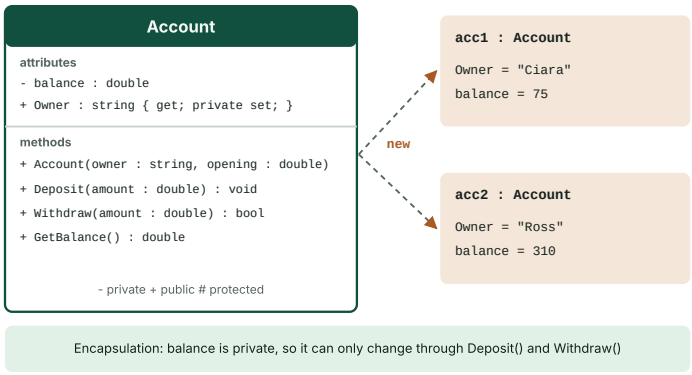
Classes, objects and encapsulation

AP24 FACT FILE

Specification: students should be able to

- Explain object-oriented programming terminology: class, object, attribute (property), method (behaviour), constructor and instantiation
- Explain the purpose of a method's parameter list and return type
- Explain encapsulation and its benefits, including private attributes accessed through get and set methods
- Write classes in C#, create objects from them and predict the output of object-oriented code
- Represent a class using a class diagram, including visibility symbols

Key words: object-oriented programming, class, object, instance, instantiation, attribute, state, method, parameter list, return type, constructor, encapsulation, private, public, get method, set method, property, class diagram



Class diagram of an Account class with private attribute and public methods, and two objects instantiated from it with their own state

Object-oriented programming: the big picture

Object-oriented programming (OOP) is a programming paradigm in which a program is organised as a collection of objects that work together. Each object bundles the data that describes it with the subroutines that act on that data, so a hire bike in a city bike-share system knows its own identifier, dock and battery level and also knows how to record a ride. This contrasts with a procedural program, where data is usually held in separate variables or records and passed into free-standing procedures and functions. The central ideas are the class, which acts as the template, and the object, which is a specific thing built from that template. Classes are made robust by encapsulation: their data is kept private and can only be reached through the methods the class chooses to make public. OOP became popular because large systems are easier to build, test and maintain when they are divided into self-contained classes that mirror real-world entities such as customers, accounts, tickets or buses. This lesson covers classes, objects, attributes, methods, parameter lists, constructors, encapsulation with get and set methods and properties, and class diagrams, using C# throughout.

Classes, objects and instantiation

A class is a template, or blueprint, that defines what every object of a particular type will contain: its attributes (the data it stores) and its methods (the behaviours it can perform). By convention a class name begins with a capital letter, for example Bike, Book or Patient. An object is one specific instance of a class. Two Bike objects share exactly the same list of attributes and the same method code, but each holds its own values, so one may be docked at City Hall with 80% charge while another sits at Titanic Quarter with 35%. The set of values held in an object's attributes at any moment is called its state. Creating an object is called instantiation, and in C# it is done with the new keyword, as in Bike b1 = new Bike("BK101", "City Hall", 80);. The variable b1 holds a reference to the new object, and its methods are then called with dot notation, such as b1.Ride(12, "Queen's University"). One class can be used to create any number of objects, which is why a well-designed class saves so much repeated code: a school library needs one Book class, not thousands of separate sets of variables.

Attributes, methods and parameter lists

Attributes, which some books call properties or fields, are variables declared inside a class. Each needs a suitable data type: a bike's ID is a string, its battery percentage is an integer, a hire price is a real number and whether a bike is reserved is a Boolean. Methods are subroutines declared inside a class that describe what an object can do or what can be done to it; the textbook also calls them behaviours. A method header shows its access modifier, return type, name and parameter list. The parameter list states what extra information the method needs to do its job, giving a type and a name for each item, so SetDock(string newDock) must be given a string, such as "Botanic", every time it is called. A method that needs no extra information has an empty parameter list, written as a pair of empty brackets, as in GetBattery(). The return type is void if the method simply carries out an action, or a data type such as int or string if it sends a value back to the calling code. The values actually supplied in a call are called arguments, and they must match the parameters in number, order and type.

Constructors

A constructor is a special method that runs automatically when an object is instantiated, and its job is to give the new object a sensible starting state. In C# a constructor has exactly the same name as its class and no return type at all, not even void, which is how it can be recognised in code and in class diagrams. Its parameters usually supply initial values that are copied into the private attributes, for example public Bike(string id, string startDock, int charge). If a parameter has the same name as an attribute, the keyword this identifies the attribute, as in this.name = name;. If a programmer writes no constructor at all, C# supplies a default one with no parameters, which leaves attributes at default values such as 0, false or null. A class may have more than one constructor provided their parameter lists differ, letting objects be created with different amounts of starting information. Constructors are also the natural place to set attributes that must never change afterwards, such as a membership number: the value is assigned once when the object is built and, because no set method is provided, it stays fixed for the life of the object.

Encapsulation with get and set methods

Encapsulation means combining an object's data and the methods that operate on that data into a single unit, the class, and hiding the internal details so that the data can only be read or changed through methods the class provides. In C# the attributes are declared private, which stops code outside the class accessing them directly. Public get methods, or getters, return the current value of an attribute, and public set methods, or setters, change it. This gives the class control over every change. A setter can validate the new value, for instance rejecting a negative age, so an object can never be put into an invalid state. An attribute that should never change once it has been allocated, such as a National Insurance number or a hospital number, is given a getter but no setter, which makes it read-only. A setter can also carry out related tasks, such as recording a change of address in an audit log, so programmers who use the class do not have to remember those side effects every time. Because other code depends only on the public methods, the internal storage can later be redesigned without breaking the rest of the program. C# properties provide the same controlled access in a more compact form.

Class diagrams

A class diagram is a design notation, taken from the Unified Modelling Language (UML), that shows a class as a rectangle divided into three compartments. The top compartment holds the class name, the middle compartment lists the attributes with their data types, and the bottom compartment lists the methods with their parameters and return types, for example + Ride(minutes : int, endDock : string) : void. Visibility is shown by a symbol in front of each member: a minus sign for private, a plus sign for public and a hash for protected. In a well-encapsulated class most attributes carry a minus sign, while the methods that form the public interface carry a plus sign. The constructor is listed among the methods, with the class's name and no return type. Class diagrams are drawn during design, before coding, so that a team can agree what each class will store and do and how the classes relate to one another, and they then serve as documentation for anyone maintaining the program later. Books and papers vary slightly in style, for instance writing 'return int' instead of ': int', so what matters is that names, data types and visibility are clear and consistent.

T Key terms from the mark schemes

Class
A template or blueprint that defines the attributes and methods (behaviours) shared by all objects of one type.
Credited: Basic building block of OOP specifying properties and behaviours

Object
A specific instance of a class, with its own values for the class's attributes.
Credited: A member of a class

Instantiation
Creating a new object from a class, in C# using the new keyword, which calls the constructor.

Attribute
A data item (field) declared in a class whose values describe the state of each object.
Credited: Property

Method
A subroutine defined in a class that describes an action an object can perform or that can be performed on it.
Credited: Behaviour

Constructor
A special method with the same name as the class and no return type that runs when an object is created to initialise its attributes.

Encapsulation
Combining data and the methods that act on it in a class, and hiding the data (private) so it can only be accessed through the class's methods.
Credited: Information hiding

Get method
A public method that returns the current value of a private attribute.
Credited: Getter / accessor

Set method
A public method that takes a new value as a parameter and uses it to change a private attribute, often after validating it.
Credited: Setter / mutator

Parameter list
The part of a method header, in brackets, that states the data items (with types) the method must be given when it is called.

Property (C#)
A class member with get and/or set blocks that gives controlled access to a private field.

Class diagram
A UML diagram showing a class as a box with its name, attributes and methods, using +, - and # for public, private and protected.

C C# example

Worked example: a bike-share Bike class

```
Bike b1 = new Bike("BK101", "City Hall", 80);
Bike b2 = new Bike("BK102", "Titanic Quarter", 35);
b1.Ride(12, "Queen's University");
b2.Ride(40, "City Hall");             // refused: not enough charge
Console.WriteLine(b1.Report());       // BK101 at Queen's University, battery 68%
Console.WriteLine(b2.Report());       // BK102 at Titanic Quarter, battery 35%
class Bike
{
    private string bikeId;
    private string dock;
    private int battery;
    public Bike(string id, string startDock, int charge) { bikeId = id; dock = startDock; battery
= charge; }
    public string GetBikeId() { return bikeId; }     // getter only: ID never changes
    public void Ride(int minutes, string endDock)
    {
        if (minutes > battery) return;              // 1% charge per minute
        battery -= minutes; dock = endDock;
    }
    public string Report() { return bikeId + " at " + dock + ", battery " + battery + "%"; }
}
```

Exam focus

Definitions are a reliable source of marks when both halves are present. A class is a template or blueprint that defines the attributes and methods (behaviours) of a type of object; an object is a specific instance of a class with its own attribute values. Questions that say 'by referring to the code' expect you to name the actual class, objects, attributes and methods from the listing, so quote identifiers such as Bike, b1, battery or Ride rather than giving only a general definition. For encapsulation, a full answer says that data and methods are held together in the class, that the attributes are made private, and that they can only be accessed or changed through get and set methods. When you write a class, examiners look for private attributes with sensible data types, a constructor that assigns every parameter to an attribute, getters that return values, setters that take a parameter and return nothing, and a display method if one is asked for. Watch for attributes that must not change, such as a student or membership number, and give them no setter. In output questions, work through each statement in order and copy the spaces, punctuation and line breaks exactly. A common slip is giving a constructor a return type.

? Check yourself

1. State the two things that a class definition specifies for each of its objects. [2]
2. A school library in Dungannon is building a loans system in C#. Using examples from this system, explain the relationship between a class and an object. [3]
3. State the purpose of a constructor and describe two ways in which a constructor differs from other methods in C#. [3]

Answers: 1. The attributes / properties (data) of the objects (1) / the methods / behaviours that can be performed on the objects (1) 2. A class is a template / blueprint defining attributes and methods (1) / an object is a specific instance / member of the class with its own attribute values (1) / suitable example, e.g. class Book; objects are individual books such as one copy with accession number A0041 (1) 3. Purpose: to initialise a new object / set the starting values of its attributes when it is created (1) / Any two from: it has the same name as the class (1) / it has no return type, not even void (1) / it runs automatically when new is used (1)